R7021E Advanced Robotics and AI

Lab 4: Grid-FastSLAM

Group 6

Dominic Addo (domadd-2) and Abdalrahman Nasser (abdnas-2)

Video evidence at:
https://drive.google.com/drive/folders/1NvlVXJmHIOX1k1Sd94HiTiOu77_A_qaz?usp=drive_link

1. Task 1: occupancy mapping with known poses

The first map was entirely unknown. The shipped log_odds_limit was zero, so clamping returned every cell to the prior. We changed one parameter at a time in Gazebo, using the same maze-drive bag and a fixed pose input.

Step	Change	Observed result
Shipped	No changes	100% unknown map.
1	log_odds_limit: 0 to 5	Cells changed, but p_occ = 0 added free evidence.
2	p_occ: 0 to 0.7	No walls. hit_window_cells = 0 had zero width.
3	p_free: 0 to 0.4	Free evidence weakened, but walls were still absent.
4	hit_window_cells: 0 to 2	Some walls appeared, covering 0.63 m2.
5	map_resolution: 0.01 to 0.05 m; map_size_m: 50 to 20 m	Walls appeared, covering 5.16 m2. Memory fell from about 100 MB to 0.6 MB per particle.
6	use_clamping: true to false	Almost no change over the 3.5-minute drive.

The tuned map still has double walls because it uses corrupted odometry. The mean pose error was 0.47 m in the Gazebo run. The 1 cm and 5 cm maps also used different noisy trajectories: the node adds noise for each received odometry message, and messages can be dropped while mapping.

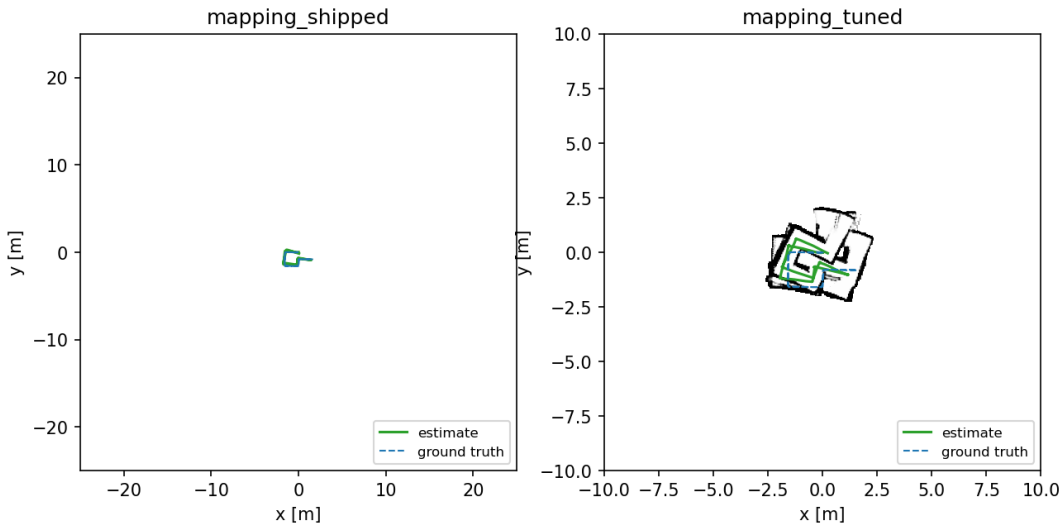


Figure 1: Task 1 maps using the shipped and tuned occupancy parameters.

2. Tasks 2 to 6: implemented models

These tasks were code exercises. The five student modules were checked before the Gazebo runs, and all checks passed.

Task	Implementation
2	The motion model samples rot1, translation and rot2, and calculates the matching log density.
3	The likelihood-field measurement model combines hit and random beam probabilities and sums their logarithms.
4	The resampling module calculates N_eff = 1 / sum(w^2) and uses systematic resampling.
5	The proposal evaluates 27 poses on a 3 by 3 by 3 lattice, weights them by measurement likelihood and motion density, and fits a Gaussian.
6	The RBPF step predicts, scan-matches, proposes or falls back, weights particles, updates maps, normalizes and conditionally resamples.

3. Task 7: Evaluate the filter in simulation

The 218 s maze-drive bag covered 13.9 m in the 4 m Gazebo maze. We used Gazebo /ground_truth for the reference pose because /odom was already 0.17 m from it. The tuning runs changed one parameter at a time.

The update gate was the first important change. Updating every 0.05 m or one degree gave 559 steps and 0.309 m ATE RMSE. Updating every 0.2 m or 0.2 rad gave 136 steps and 0.085 m. Wider odometry sigmas were also needed to cover the motion error in each larger step. The final simulation settings used odometry_sigmas = [0.05, 0.04, 0.05].

A scan-matching threshold of 0.55 was needed because an unknown map cell has an occupancy probability of 0.5. A threshold at or below 0.5 would accept matches against an empty map instead of falling back to odometry.

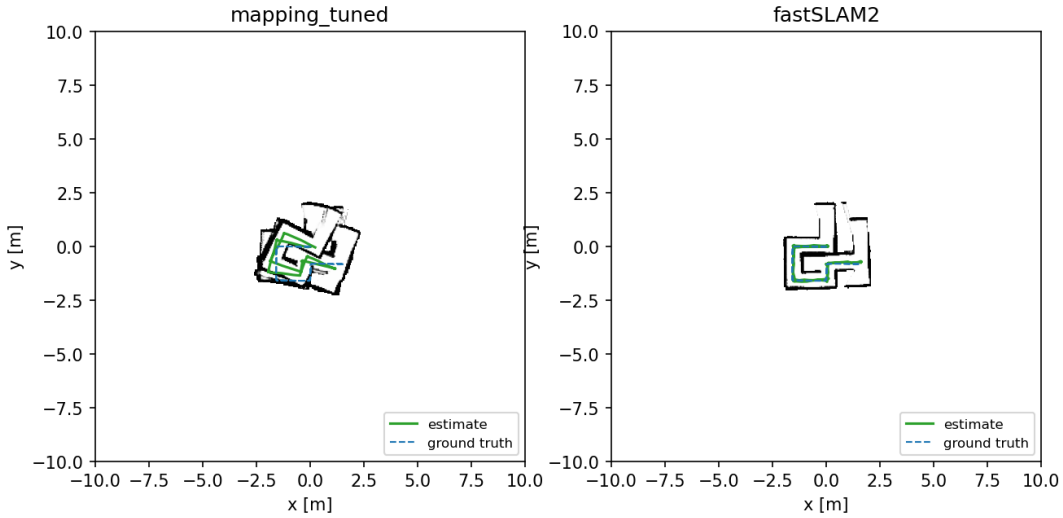


Figure 2: Task 7 map comparison. The FS2 estimate has single walls where raw odometry produced doubled walls.

Particle count and error

The ATE sweep used three seeds for each particle count. FS2 reduced error at low particle counts. From N = 10 the result ranges overlapped, so the runs do not show a consistent advantage from adding more particles.

N	FS1 mean [m] (range)	FS2 mean [m] (range)
1	1.081 (0.636 to 1.683)	0.387 (0.272 to 0.476)
5	0.342 (0.184 to 0.545)	0.239 (0.199 to 0.279)

10	0.210 (0.137 to 0.283)	0.130 (0.088 to 0.197)
20	0.175 (0.082 to 0.324)	0.161 (0.070 to 0.326)
50	0.101 (0.085 to 0.131)	0.142 (0.121 to 0.180)

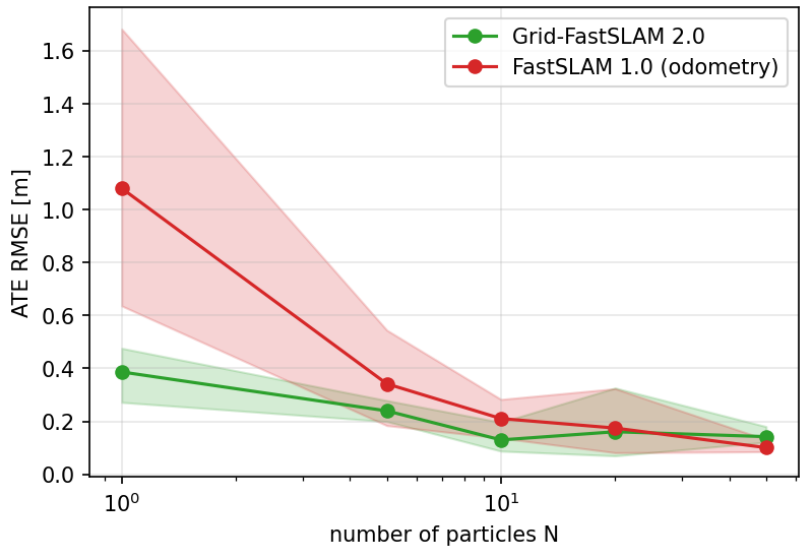


Figure 3: ATE RMSE for FS1 and FS2 across three random seeds.

Resampling and processing time

With N = 10, selective resampling still ran on 78% of steps. Median $\frac{N_{eff}}{N}$ was 0.34, below the 0.5 threshold. The likelihood is sharp because about 300 beams are treated as independent, so particle weights collapse on most steps.

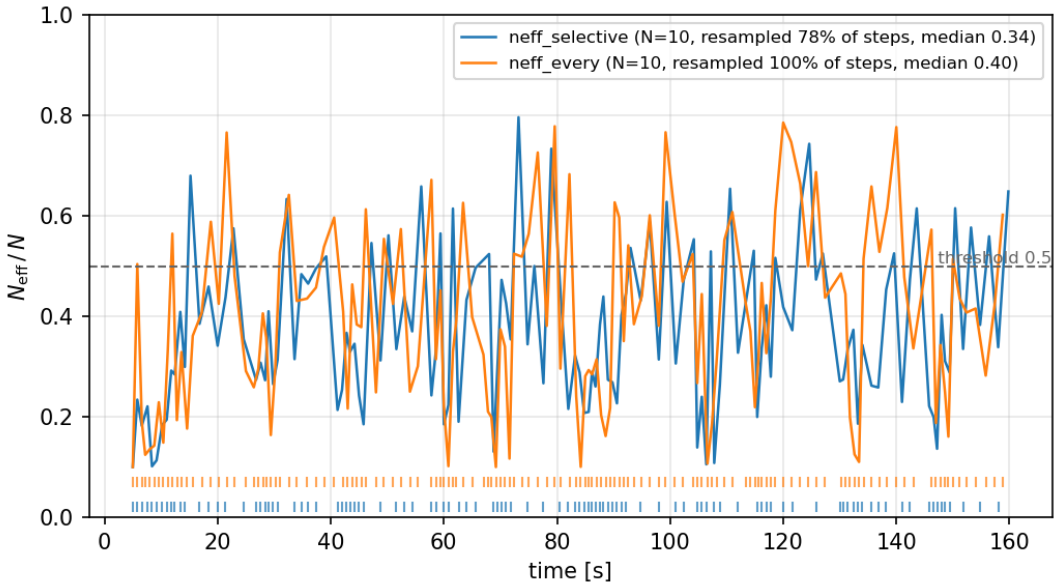


Figure 4: Effective particle count and resampling events during the sweep.

Processing time grew with particle count, at about 35 ms per step at N = 1, 230 ms at N = 10 and 830 ms at N = 50. A step must finish before the next gated scan, which can arrive in about 0.4 s during a turn. At N = 50, the filter could not keep up.

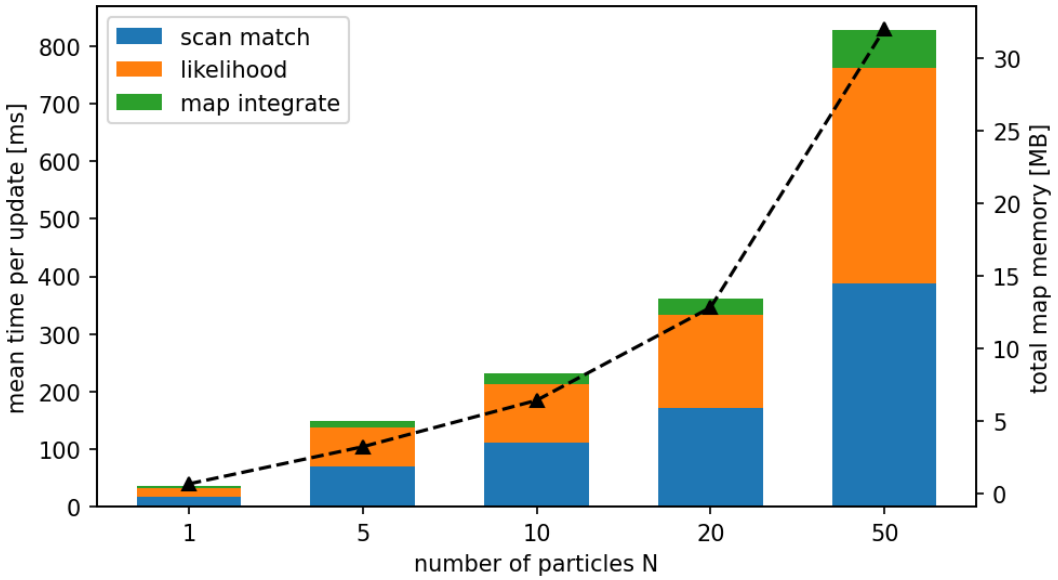


Figure 5: Mean processing time by filter stage and total map memory.

4. Task 8: transfer to the lab hall

The hall runs used FS2, N = 10, odom_noise_sigma = [0, 0, 0], scan_queue_depth = 1 and no ground truth. The run notes initially said N = 20 for some recordings, but each saved run configuration confirms N = 10. The tuned odometry sigmas were [0.03, 0.02, 0.06]. The Task 8 run 1 and Task 9 RViz screen recordings and the Task 8 run 2 bag-replay video are included separately. The run 2 filter was rerun on the saved sensor data, so its particle samples differ from the live run.

Run	Steps	Time [s]	End gap, estimate / odometry [m]	Fallbacks per step	Median step [ms]
Official run 1, full maze	93	282	1.90 / 2.01	0.11	112
Official run 2, two loops	146	320	0.38 / 0.42	0.08	76

Run 1 did not return to the start. The estimate and odometry both ended about 2 m away, and the bag was shorter than the run log. Run 2 stopped about 0.4 m from the start line. Both driving maps have single, sharp walls. The estimate stayed within 0.04 to 0.12 m of odometry at the end. At N = 10, a filter step took about 76 ms in most runs, well below the time between updates.

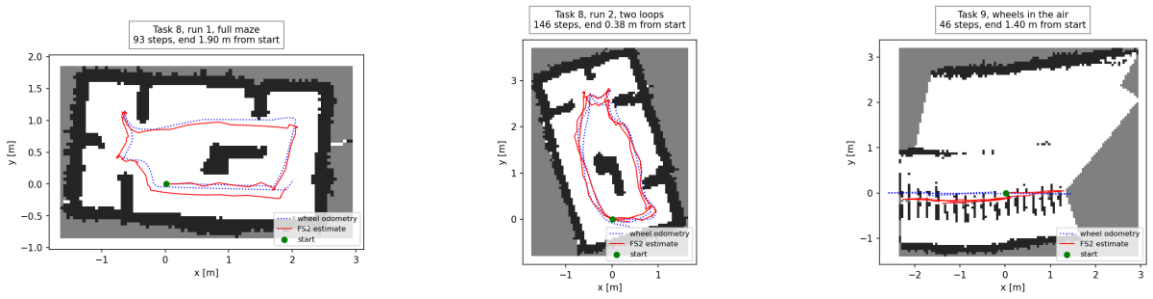


Figure 6: Lab hall results for the two Task 8 runs and the Task 9 wheels-in-the-air run.

The robot odometry performed better than the simulation odometry with 0.003 noise added. Changing N or the motion sigmas did not visibly change the hall maps. With good odometry, the particles started close to the pose, and the filter had less error to correct.

5. Task 9: wheels in the air

The wheels turned while the robot stayed in place. Odometry reported 9.3 m of back-and-forth travel and an end gap of 1.72 m. The filter estimated 8.8 m of travel and an end gap of 1.40 m. On a typical step, the estimate followed 95% of the odometry increment. The map smeared because scans of the same walls were added at successive incorrect poses.

There were 5.5 fallback particles per step on average, compared with 0.1 when driving normally. All 10 particles fell back on 13 of 46 steps. N_{eff} averaged 3.5 of 10, and resampling ran on 38 of 46 steps.

The failure follows the filter sequence. The odometry gate sees 0.2 m of motion and starts a step. The motion model moves each particle forward. The scan matcher stays near that prediction because a 0.2 m correction costs 5 in its prior, while the measurement score is at most 1. The proposal lattice only searches +/-0.05 m, so it cannot reach the real stationary pose either. Weighting and resampling cannot choose a pose that no particle represents.

$$prior\ cost\ =\ 0.1\ *\ 0.5\ *\ \left(\frac{0.2\ m}{0.02\ m}\right)^2\ =\ 5$$

From what we see, the filter believes one odometry step is wrong by about 2 cm, not 20 cm, so it does not search far enough to undo the wheel slip.

Comparison with other SLAM filters

FastSLAM 1.0 samples only from the motion model. Every particle would still follow the false odometry step, with some spread around it. The weights might prefer the particle that moved least, but none would stay at the real pose. It would fail the same way, only without the scan-matching proposal.

An EKF-SLAM filter predicts from odometry and updates the pose from landmark measurements through its Kalman gain. It could correct part of each false step, but would still drift if odometry covariance were too small or data association were wrong. In the lab runs, the filter tracked good odometry, but wheel slip drove the particles away from the stationary pose.